

可以，我们就按“**屏幕中心瞄准** → **按住左键抓取** → **随相机移动** → **松开后放下**”来实现。结合你现在的代码，建议按下面顺序完成，每一步都先验证再继续。

1. 让两个物体的位置成为持久变量。

你现在生成 `MA`、`MB` 时，每帧都直接写入固定的目标位置。先把两个球的**世界空间球心位置**保存到 `while` 外面的变量中，再用这些变量生成模型矩阵。

验证：修改其中一个位置变量，只对应的球发生移动；移动相机时，两个球仍然固定在世界中。

2. 记录左键按下、持续按住和松开。

增加鼠标按键回调，区分左键的按下和释放。现有的 `MouseButtonCallback` 处理的是鼠标移动，需要另一个回调处理按键。[GLFW 鼠标按键说明](#)

还需要保存“当前选中哪个物体”的状态，并能表示“没有选中任何物体”。

注意：你已经通过窗口的 `user pointer` 保存 `MouseButton`，新增状态要与它统一组织，不能直接替换指针而让原来的回调读取错误类型。

验证：一次点击只触发一次按下和一次松开，按住期间状态保持。

3. 构造屏幕中心对应的射线。

射线 (ray) 可以理解为：从相机出发，沿准星方向向前寻找物体。

对你现在的相机：

- 射线起点是世界空间中的 `eye`。
- 射线方向是归一化后的 `direction`。
- 不需要读取鼠标在窗口中的位置，因为准星始终在屏幕中心。

验证：转动视角后，射线方向跟着变化；方向长度保持为 1。

4. 判断射线击中了哪个物体。

先针对当前两个球学习射线与球的相交判断。使用每个球的**当前世界空间球心**和半径，判断是否命中，并比较命中的距离。

选择规则：

- 没有命中：不抓取任何物体。
- 命中一个：选择它。
- 命中多个：选择相机前方最近的命中物体。

你的球网格是三角形近似球，因此用球形包围体点选属于近似判断；以后支持任意模型时，可以再考虑射线与三角形相交。

验证：先打印选中物体的编号，暂时不要移动它。测试瞄准空白、两个球分别被瞄准，以及两个球前后重叠的情况。

5. 在抓取开始时，记录物体相对相机的位置。

只在左键刚按下、成功选中物体时记录一次。

要保存的是物体球心在**相机坐标空间**中的位置。这样才能表达“在视野右侧多少、上方多少、前方多远”。

仅保存世界空间里的 物体位置 - `eye`，只能跟随相机平移，无法正确跟随相机转头。

验证：按下左键瞬间，物体不能突然跳到准星中心。

6. 按住期间更新被抓物体，松开后保留最终位置。

每帧先更新相机，再根据第 5 步保存的相机空间位置，求出被抓物体的新世界空间位置。只更新被选中的物体。

左键松开时，清除抓取状态，保留物体最后的位置。不要恢复最初的坐标。

验证：

- 按住并前进、后退、平移：物体与相机的相对位置不变。
- 按住并转头、抬头：物体在视野中的相对位置保持。
- 松开后继续移动相机：物体留在放下的位置。

7. 最后添加可见准星。

先完成点选和抓取逻辑，再画屏幕中央的小十字或圆点。准星绘制不应受场景深度遮挡。

整个循环最终要遵循：**处理输入** → **更新相机** → **处理抓取与物体位置** → **生成矩阵** → **绘制**。回调可以先记录按键事件，让主循环用更新后的相机处理点选。

你现在先完成第 1 步：把两个球的目标位置从 `translation(...)` 的固定参数中提取出来，放到 `while` 外面。